

Indian Institute of Information Technology, Kota

ECD302 - Skill Transformation Based Engineering Project - I

Hardware Level Atomic Instruction for Source-Authenticated Sensor Data Acquisition

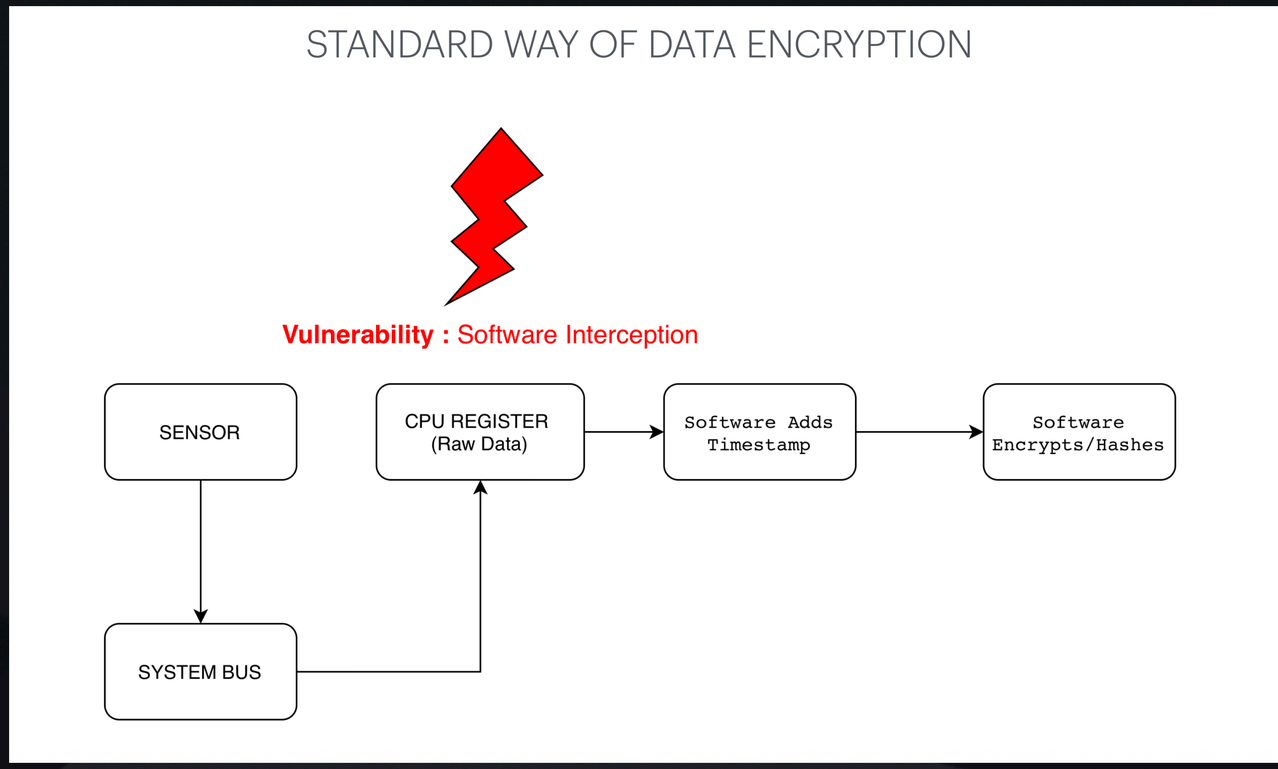
50% Implementation Milestone and Silicon Simulation

Submitted By : Gyan Veer Singh, Chinmay Nirala, Garapati Mohith, Vadekar Baburao
Guided By : Dr. Ashok Kumar Kherodia

Core Problem

Recap

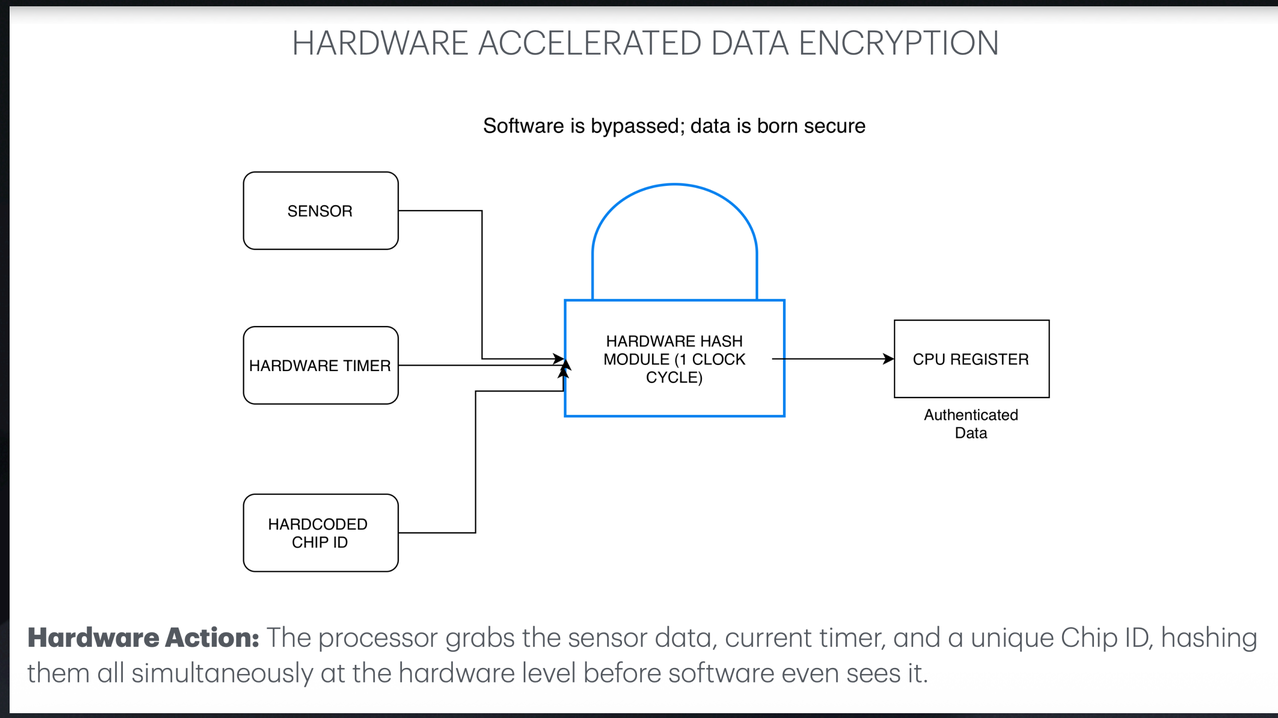
- **The Threat :** Standard systems read raw data into memory, leaving a window where hackers can intercept or alter it via bus snooping before software can encrypt it.
- **The Flaw:** Traditional software-based encryption is too slow and occurs too late in the pipeline.
-



Custom Architecture

Recap

- **The Solution:** Move the security perimeter entirely into the hardware datapath.
- **The Implementation:** We are modifying the open-source RISC-V IBEX32 core, injecting a custom atomic instruction (LDSENS).
- **The Guarantee:** The data is hashed natively in the hardware in a single clock cycle before it ever touches the system bus.



Current Progress

Instruction Decode

Modified ibex_decoder.sv

- We successfully intercepted the RISC-V custom-0 opcode space within the Ibex decoder.

```
// CUSTOM LDSENS Instruction
// CUSTOM LDSENS Instruction
7'b0001011: begin
  illegal_insn_o     = 1'b0;
  rf_we_o            = 1'b1;  // destination register to write enable

  alu_op_a_mux_sel_o = OP_A_REG_A;  // send rs1 to ALU input A
  alu_op_b_mux_sel_o = OP_B_IMM;  // send immediate to ALU input B
  imm_b_mux_sel_o    = IMM_B_I;  // use I type immediate formatting

  alu_operator_o     = ALU_LDSENS;
  rf_wdata_sel_o     = RF_WD_EX;

    Remove trailing spaces. [Style: trailing-spaces][no-trailing-spa

end
```

Custom Cryptographic Hash Module

ibex_crypto_engine.sv

- Designed the ibex_crypto_engine.sv hash module which operates independently while fetching sensor data through LDSENS instruction.
- Uses XOR Shift Hashing : $\text{Secure Payload} = \text{SILICON_KEY} \oplus \text{Hardware_timer} \oplus \text{Sensor_data}$
- This is a purely combinational circuit due to the nature of hashing algorithm. Hence, executes in a single clock cycle.

Ibex_crypto_engine.sv

```
module ibex_crypto_engine (
    input logic clk_i,
    input logic rst_ni,


    input logic        en_ldsens_i,          // enable ldsens so cpu knows data is being fetched_
    input logic [31:0] direct_sensor_data_i,

    output logic [31:0] secure_payload_o

);
```

Silicon Key burned in to the hardware

```
  // BURNED SILICON KEY_

  localparam logic [31:0] SILICON_KEY = 32'hA52C_BC83;


  // cycle accurate timer : immune to software interrupts_

  logic [31:0] hw_timer_q;

  always_ff @(posedge clk_i or negedge rst_ni) begin
    if (!rst_ni) begin
      hw_timer_q <= 32'b0;
    end else begin
      hw_timer_q <= hw_timer_q + 1'b1;
    end
  end


  // ATOMIC HASHING LOGIC (COMBINATIONAL) -> XOR SHIFT HASHING
  //


  always_comb begin
    secure_payload_o = 32'b0;

    if (en_ldsens_i) begin
      secure_payload_o = direct_sensor_data_i ^ SILICON_KEY ^ hw_timer_q;
    end
  end

endmodule
```

Purely Combinational

C++ Simulation Environment

Tools : Verilator

- **The Challenge:** We needed a way to test raw silicon logic before flashing an FPGA.
- **Our Solution:** We built a custom C++ driver using Verilator to act as a virtual motherboard.
- **Execution:** We successfully "hot-wired" the CPU, feeding it the raw hexadecimal machine code for LDSENS (0x0000050B) and simulating the clock cycles.

```
x_decoder.sv 4 ●   ibex_crypto_engin… 8   sim_main.cpp 7
#include "Vtop_sim.h"      'Vtop_sim.h' file not found
#include "verilated.h"
#include "verilated_vcd_c.h"

int main(int argc, char **argv) {
  Verilated::commandArgs(argc, argv);      Use of undeclared identifier 'Verilated
  Verilated::traceEverOn(true);      Use of undeclared identifier 'Verilated'

  Vtop_sim *top = new Vtop_sim;      Unknown type name 'Vtop_sim'
  VerilatedVcdC *tfp = new VerilatedVcdC;      Unknown type name 'VerilatedVcdC'

  top->trace(tfp, 99);
  tfp->open("waveform.vcd");

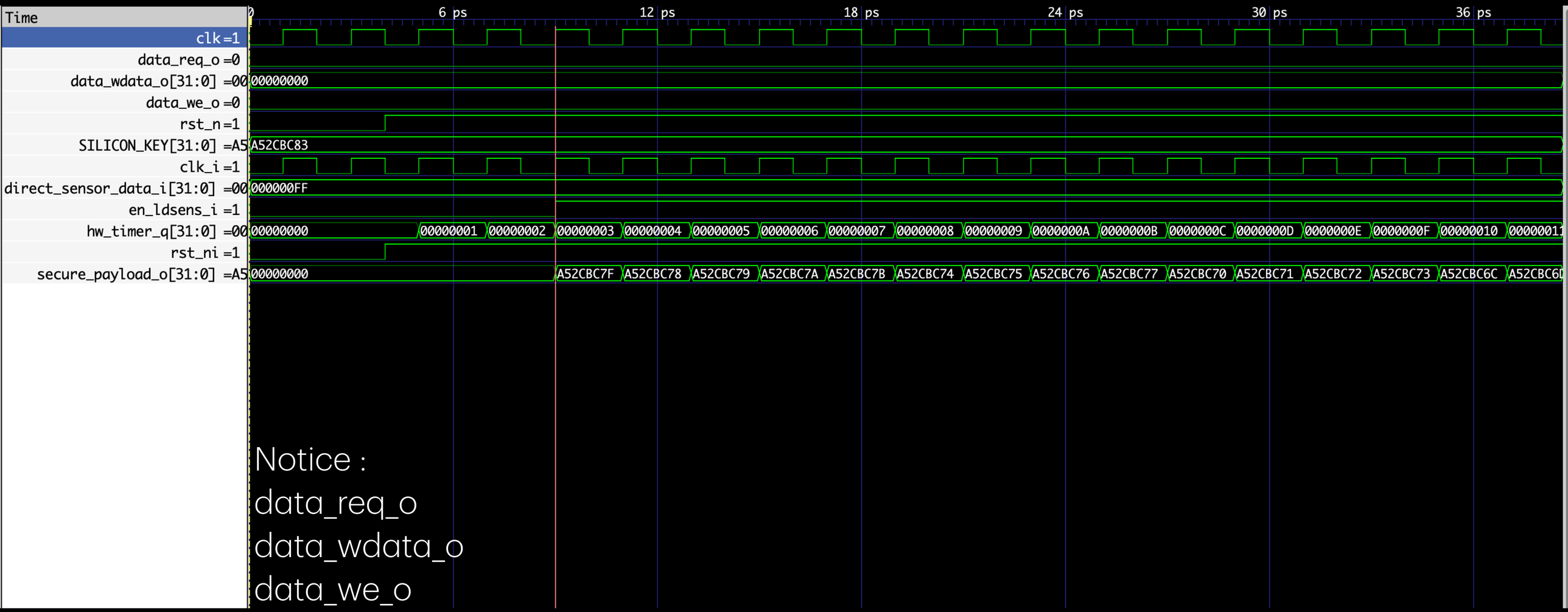
  for (int time = 0; time < 40; time++) {
    top->clk = time % 2;
    if (time < 4)
      top->rst_n = 0;
    else
      top->rst_n = 1;

    top->eval();
    tfp->dump(time);
  }

  tfp->close();
  delete top;
  return 0;
}
```

Simulation Result

GTK WAVE : WAVEFORM OF MODULE



Notice :

data_req_o

data_wdata_o

data_we_o

Future Milestones

Verification Instruction

VRFSENS

- **Custom Instruction VRFSENS :** The instruction will enable us to verify the data and decrypt it.
    - **The Goal:** Allowing the system to read the hashed data from RAM and cross-reference it against the hardware timer to flag tampering.

FPGA Integration

- After the competition of the project the next steps would be to burn it on to the FPGA and show real life examples.

Expanding Security Scopes

- **The Known Limitation:** LDSENS prevents software spoofing, but the physical wire from the sensor to the CPU is still vulnerable to Man-in-the-Middle attacks.
- **Future Architecture:** Proposing an asymmetric signature verification module right at the physical input pins for true end-to-end Zero-Trust. Enabling monitoring of latency through which man in the middle hardware spoofing attacks can be mitigated.

REFERENCES

1.IEEE Transactions on Computers (2024): Y. Chen, X. Wang, S. Song, L. Feng, and Z. Wang, *"RISC-V Custom Instructions of Elementary Functions for IoT Endpoint Devices,"* IEEE, vol. 73, pp. 523-535. DOI: 10.1109/TC.2023.3338829.

2.MDPI Electronics (2024): B. Kieu-Do-Nguyen, K. Nguyen, et al., *"A Trusted Execution Environment RISC-V System-on-Chip Compatible with Transport Layer Security 1.3,"* MDPI, vol. 13, no. 13. DOI: 10.3390/electronics13132508.

3.OpenTitan Project: Google & lowRISC, *"OpenTitan: Open Source Silicon Root of Trust,"* [Online]. Available: https://opentitan.org

4.Technology Innovation Institute (2023): *"Zero Trust Secure RISC-V System,"* Secure Systems Research Centre Whitepaper.